

COVER STORY

이진원

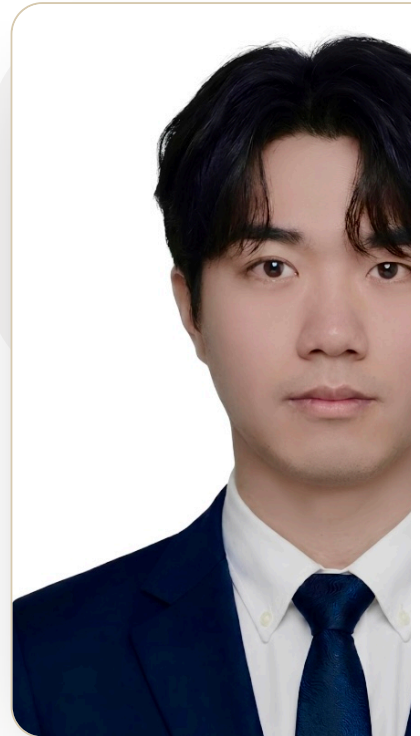
— Lee Jinwon

Edition Four — Building across layers.

Full-stack Engineer · Mobile · Backend · Embedded Sim

ljin2091@naver.com

GitLab ↗ (<https://lab.ssafy.com/ljin2091>)



PORTRAIT · 2026

ROLE

Full-stack Engineer

FOCUS

End-to-end systems

BASED

Seoul, KR

AVAILABLE

2026.06 +

About

Flutter 앱부터 Spring/Django 백엔드, Isaac Lab 시뮬레이션과 라즈베리파이 디바이스 통합까지 — 한 프로젝트의 끝에서 끝까지 책임지는 풀스택 엔지니어를 지향합니다.

#풀스택 통합

#디바이스-서버-앱 연동

#RL 시뮬레이션

#인프라 자동화

ROLE Full-stack Engineer 지향

INTEREST End-to-end product delivery, Cross-layer integration, Reinforcement learning sims

MOTTO *“Build end-to-end. Bridge the gaps. Ship as one.”*

Tech Stack

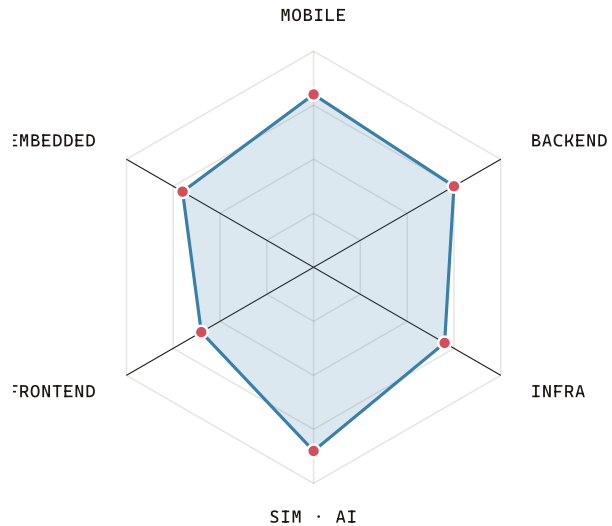
프로젝트에서 실제로 손에 익은 도구들.

STRENGTH MAP

어떤 영역에 무게가 실렸나

네 개의 프로젝트에서 실제로 손에 익은 정도를 분포로 본 것. 절대 점수가 아니라 어디에 시간을 쏟았나의 그림에 가깝다.

- **MOBILE** — Flutter · IMO 앱 아키텍처/계층 통합
- **BACKEND** — Spring Boot · Django DRF · API 명세 정합
- **INFRA** — Docker · Jenkins · Caddy · Nginx · EC2
- **SIM · AI** — Isaac Lab · RL navigation · 멀티 도킹
- **FRONTEND** — Vue.js · WebRTC 시그널링 · UI 통합
- **EMBEDDED** — Raspberry Pi · 앱-디바이스 핸드셰이크



Languages

프로젝트에서 주력으로 쓴 언어



Python



Dart



Java



TypeScript



JavaScript

Frontend & Mobile

앱·웹 클라이언트



Flutter



Vue.js



React

Backend

API·서비스 서버



Spring Boot



Django



Django REST Framework



Node.js

Realtime · Data

실시간 통신과 데이터 저장



WebSocket



WebRTC



Redis



MySQL



PostgreSQL

Infra & DevOps

배포·운영 인프라



Docker



Jenkins



Nginx



Caddy

Sim · AI · Embedded

특화 기술 — 시뮬레이션부터 디바이스까지



Isaac Lab



Reinforcement Learning



ROS



Raspberry Pi

Tools

협업·이슈·디자인



Git



GitLab



Figma



Jira



Mattermost



Notion

Stories

가장 무게 있게 작업한 네 편의 이야기.

STORY NO. 01

2026.04 - 2026.05 · 6주 · 자율PJT

IMO

스마트 글래스로 운동 자세를 보고, 라즈베리파이에서 추정된 데이터를 앱이 받아 통계·코치로 돌려주는 풀스택 운동 코치. 디바이스-서버-앱 3개 계층을 사용자 의도에 맞게 묶는 게 과제였다.

46건

MERGED MR

3계층

E2E 통합

2환경

로컬/운영 배포 분리

 Flutter  Spring Boot  Django  Redis  Docker  Raspberry Pi +2[read the story →](#)

STORY NO. 02

2026.02 - 2026.04 · 6주 · 특화PJT

AMR 시뮬레이션 플랫폼

Isaac Lab 위에서 자율이동로봇(AMR) 다대수가 동시에 학습·도킹·서비스 사이클을 도는 시뮬레이션 환경. 단일 로봇 학습을 6/12/18대 스케일링까지 끌어올리는 게 과제였다.

50건

MERGED MR

6 → 18대

스케일링 폭

3단계

도킹 이후 서비스 사이클

 Isaac Lab  RL  Python  ROS  Docker  Jenkins +2[read the story →](#)

STORY NO. 03

2026.01 - 2026.02 · 6주 · 공통PJT

itda

WebRTC로 다인 협업을 묶고 통합 대시보드 UI에서 룸·타임라인·세션을 한 번에 관리하는 AI 영상 제작 협업 플랫폼.

3채널

실시간 협업 구조

6명

WEBRTC MESH 통화 범위

공통 셀

사이드바 + MINITIMELINE 통합

Vue.js

WebRTC

Spring Boot

Node.js

Docker

S3

read the story →

STORY NO. 04

2025.12 · 1주 · 관통PJT

MovieApp

영화 추천·한줄평·마이페이지를 갖춘 영화 커뮤니티. 첫 풀스택 협업 프로젝트로, 사용자 콘텐츠 흐름(좋아요·한줄평·나중에 볼)을 끝에서 끝까지 묶는 게 과제였다.

4흐름

직접 만든 사용자 기능

5건

MERGED MR (1주)

1주

관통PJT 기간

Vue.js

dj Django REST Framework

TMDB API

read the story →

Notes

2025.12 ~ 2026.05

SSAFY 14기

교육생 · 4개 프로젝트 완수

- 관통PJT — 영화 커뮤니티 (마이페이지·한줄평·추천)
- 공통PJT itda — 화상 협업 학습 플랫폼 (WebRTC·UI 통합)
- 특화PJT AMR Sim — 다중 AMR 자율주행 시뮬레이션 (Isaac Lab/RL·인프라)
- 자율PJT IMO — 스마트 글래스 운동 코치 (Flutter·백엔드·라즈베리파이 통합)

Awards

2026 SSAFY 14기 공통PJT

우수상 (1등)

itda — AI 영상 제작 협업 플랫폼

Education & Certs

EDUCATION SSAFY 14기 (Samsung SW Academy For Youth)
2025.07 – 2026.05 수료 예정

수료 예정

Contact

새로운 팀, 새로운 문제,
새로운 계층의 통합 —

어디든 끝에서 끝까지 책임지고 싶습니다.

Email →

GitLab ↗ (<https://lab.ssafy.com/ljin2091>)

IMO

스마트 글래스로 운동 자세를 보고, 라즈베리파이에서 추정한 데이터를 앱이 받아 통계·코치로 돌려주는 풀스택 운동 코치. 디바이스-서버-앱 3개 계층을 사용자 의도에 맞게 묶는 게 과제였다.

Flutter

Spring Boot

Django

Redis

Docker

Raspberry Pi

WebSocket

Gemini API

01 BACKGROUND

PROBLEM

운동 코칭 앱은 많지만, 동작 자세를 "본다"는 부분은 사용자가 직접 영상을 찍어 올리는 데 의존한다. 그러다 보니 한 세트가 끝날 때마다 동작이 끊겨 운동에 몰입하기 어렵고, 코칭 데이터도 그때그때 단발성으로만 남는다. 우리는 사용자가 운동 중에 시선을 떼지 않고도 자세 피드백·통계·코치 응답을 받을 수 있어야 한다고 봤다.

OUR APPROACH

스마트 글래스(촬영) → 라즈베리파이(자세 추정) → 백엔드(세션·통계·챗봇) → Flutter 앱(피드백·기록) 4계층을 한 흐름으로 묶어, 사용자는 글래스만 쓰고 있으면 나머지가 자동으로 굴러가는 운동 사이클을 만들기로 했다.

EVIDENCE · NOTES

- 홈트레이닝/헬스장 사용자가 정확한 자세 피드백 없이 반복 운동을 이어가면 부상·정체로 이어진다는 관찰
- 운동 중에는 몸을 점검할 손이 없다.

- 기존 운동 앱들은 한 세트가 끝날 때마다 사용자가 직접 영상을 찍거나 입력해야 해서, 흐름이 끊기고 누적 기록이 들쭉날쭉했다.

02 SNAPSHOT

MERGED MR

46건

Flutter · backend ·
device 작업 반영

E2E 통합

3계층

Flutter ↔ backend ↔
Raspberry Pi

로컬/운영 배포 분리

2환경

Docker Compose 분기로
Redis 노출 격리

03 ROLE

TEAM

5인 팀 (앱·백·디바이스 분
담)

MY SCOPE

Flutter 앱 초기 아키텍처, 백엔드 세션/통계 API, 라즈베리파
이 ↔ 앱 실시간 핸드셰이크 프로토콜, Redis/Docker 환경
통합

04 VISUALS

대표 흐름 보드

PREVIEW

실제 앱 캡처가 남아 있지 않아, 포트폴리오용으로 핵심 사용자 흐름을 다시 정리한 보드.

 IMO 앱의 온보딩, 운동 시작, 통계/챗봇 흐름을 요약한 다이어그램

캐릭터 자산

SCREEN

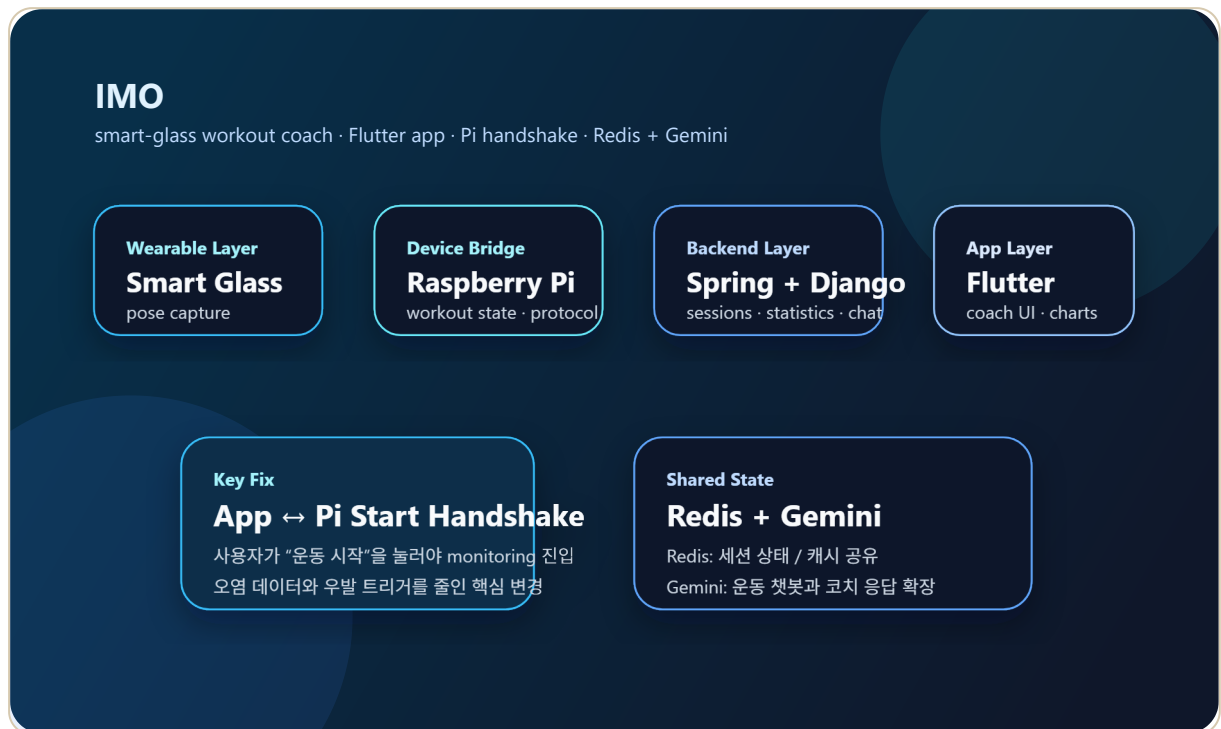
운동 종료 후 결과 대기 흐름에 활용한 대표 캐릭터 자산. 앱의 친화적 톤을 만드는 요소였다.



시스템 아키텍처

스마트 글래스, Pi, 서버, 앱, Redis/Gemini가 어떻게 연결되는지 포트폴리오용으로 재구성한 아키텍처.

ARCH



05 PROBLEM SOLVING

문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

01

디바이스가 사용자 의도를 무시하고 자동 진입했다



PROBLEM

캘리브레이션이 성공하면 라즈베리파이가 곧바로 monitoring 상태로 진입해 운동 데이터를 쌓기 시작했다. 사용자가 글래스를 아직 안 쓴 상태에서도 세션이 시작돼 누적 데이터가 오염됐다.



APPROACH

Pi 단독 결정 → 앱-Pi 양방향 핸드셰이크로 전환.
PiMessageType.startWorko 메시지를 신설해 앱에서 "운동 시작" 버튼을 눌러야만 Pi가 monitoring으로 들어가도록 흐름을 바꿨다.
WorkoutRepository 계층까지 변경 전파.



RESULT

세션이 사용자 동의 시점에만 시작되도록 바뀌어 오염 데이터가 사라졌고, 클래스 착용 전 우발 트리거를 0건으로 줄였다.

↗ MR !83 · 앱-Pi 양방향 운동 시작 핸드셰이크 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/83)

02

주간 추세 막대가 잘못된 요일에 찍혔다



PROBLEM

월요일에 운동을 3번 하면 막대가 일·월·화 자리에 1개씩 분산되어 표시됐다. 백엔드가 세션별 trend values를 순서대로 append하는 구조였고, 앱은 borderRadius=999인 막대를 그대로 쌓아 모양도 깨졌다.



APPROACH

백엔드는 일자별 7일 슬롯에 매핑하도록 집계 로직을 재설계했다. 앱은 막대 모양을 슬롯 단위로 정상화하고 같은 날 다중 세션을 한 막대로 합쳐 그리도록 수정했다.



RESULT

막대 위치와 모양이 모두 정상화되어 사용자가 주간 운동 패턴을 한눈에 읽을 수 있게 됐다.

↗ MR !101 · 주간 추세 일자별 7일 슬롯 매핑 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/101)

! PROBLEM

달력 화면은 한 달치 데이터와 마커가 필요하고, 리스트 화면은 날짜/타입 필터와 페이지징이 필요한데 기존 GET /sessions는 그걸 한 호출로 주지 못해 화면이 2~3번 호출해야 했다.

→ APPROACH

쿼리 파라미터를 spec(API-07)에 맞춰 date / month / exerciseType + page/size로 정리했다. 응답은 sessions 배열 + exerciseDates(달력 마커) + pagination을 한 번에 담은 구조로 통일했다.

✓ RESULT

달력 화면은 1회 호출로 한 달치를 받고, 리스트 화면도 동일 엔드포인트로 처리해 클라이언트 코드 분기를 줄였다.

↗ MR !25 · GET /sessions 명세 정합 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/25)

로컬 디버깅용 Redis 노출이 운영 환경에 그대로 새어 나갈 위험이 있었다

! PROBLEM

Pi·서버·앱이 세션 상태를 공유하려고 Redis를 함께 보는 구조였는데, 로컬에서는 디버깅을 위해 Redis 포트를 외부로 열어야 했다. 같은 compose 파일을 그대로 EC2에 쓰면 운영 인스턴스에서도 Redis가 외부 노출되어 무차별 접근 위험이 생긴다.

→ APPROACH

docker-compose.yml을 베이스로 두고 docker-compose.local.yml과 docker-compose.prod.yml을 override 파일로 분리했다. 로컬은 Redis 6379 포트를 호스트에 매핑하고, 운영은 동일 서비스를 내부 네트워크에만 묶이도록 설정한다. 민감 키(Gemini API key, DB 자격증명)는 .env 기반으로 분리해 이미지에 박히지 않게 했다.

✓ RESULT

같은 코드베이스/이미지를 환경 파일만 바꿔 로컬과 EC2에 안전하게 띄울 수 있게 됐고, 운영 Redis가 외부에 열린 채로 배포되는 사고를 구조적으로 차단했다.

06 TECH DECISIONS

Flutter

Android/iOS 동시 출시가 필요했고 디바이스 통신이 핵심이라 빠른 빌드-실행 주기 검증 사이클이 중요했다. React Native 대비 단일 코드베이스에서 BLE/WebSocket 같은 디바이스 채널을 안정적으로 다루기 쉬워 선택했다.

Redis (in-memory state bus)

Pi·앱·서버 3자가 운동 세션 상태를 공유해야 하는데 DB polling으로는 latency와 부하가 누적된다. in-memory key로 상태를 주고받으면 코드가 단순해지고, 세션 종료 시 TTL로 자연 만료시킬 수 있어 청소 로직이 따로 필요 없다.

Docker Compose override (local / prod 분리)

같은 이미지를 환경 파일만 바꿔 띄우려면 단일 compose로는 부족했다. base + override 패턴으로 로컬은 Redis를 노출하고 운영은 내부 네트워크에만 두는 식으로 분리하면, 배포 실수로 민감 포트가 열리는 사고를 구조적으로 막을 수 있다.

Spring Boot + Django 이원화

세션·통계·인증처럼 트랜잭션
이 분명한 영역은 Spring
Boot에, Gemini 호출과 챗봇
·LLM 응답처럼 파이썬 생태
계 자산이 풍부한 영역은
Django에 두는 게 팀의 개발
속도와 검증 자원 모두에 유리
했다.

07 OUTCOMES

운동 시작 우발 트리거

0건

사용자 버튼 기반 양방향 핸드셰이크 도입 후

달력+리스트 통합 페이지

1회 호출

달력/리스트가 같은 GET
/sessions 엔드포인트를 공유

주간 추세 집계 기준 정립

7일 슬롯

세션 append 방식 → 일자
매핑 방식으로 재설계

COMPOSE OVERRIDE 패턴

환경별 분리

로컬/EC2가 같은 이미지를
다른 노출 규칙으로 띄우도록
정리

08 SELECTED MRS

MR !83 · 앱-Pi 양방향 운동 시작 핸드셰이크 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/83)

사용자 버튼을 기준으로 monitoring 진입 시점을 제어한 핵심 변경.

MR !25 · GET /sessions 명세 정합 (API-07) (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/25)

달력과 리스트 화면을 한 엔드포인트로 맞춘 API 정리.

MR !101 · 주간 추세 7일 슬롯 매핑 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/101)

통계 집계와 앱 시각화가 어긋나던 문제를 함께 수정.

09 RETROSPECTIVE

LESSON

디바이스-서버-앱 세 계층이 얹힌 흐름은 "누가 상태의 주인인가"를 먼저 못 박지 않으면 작은 트리거 하나가 전체를 오염시킨다는 걸 핸드셰이크 이슈에서 배웠다.

NEXT TIME

다음에는 첫 주에 상태 머신 다이어그램부터 합의하고, 각 메시지가 어떤 권한을 위임하는지 코드 주석과 타입 수준에서 명시하겠다.

AMR 시뮬레이션 플랫폼

Isaac Lab 위에서 자율이동로봇(AMR) 다대수가 동시에 학습·도킹·서비스 사이클을 도는 시뮬레이션 환경. 단일 로봇 학습을 6/12/18대 스케일링까지 끌어올리는 게 과제였다.

 Isaac Lab  RL  Python  ROS  Docker  Jenkins  EC2  Caddy

01 BACKGROUND

PROBLEM

물류·서비스 현장에서 AMR을 다대수로 운용하려면 충돌 회피·도킹·서비스 사이클을 함께 학습해야 한다. 그런데 실기로 6대 이상을 동시에 돌리는 건 공간·안전·전력 비용이 크고, 알고리즘 검증 단계에서는 더더욱 부담스럽다. 시뮬레이션에서 먼저 다대수 환경을 만들어, 실기 부담 없이 정책을 검증·확장하는 길이 필요했다.

OUR APPROACH

Isaac Lab 위에서 학습 환경(rl_navigation 패키지) · 플레이 런타임(play_navigation_multi) · 배포 인프라(Jenkins/Caddy/EC2)를 한 라인 위에 정리해, 6/12/18대 스케일링 실험과 도킹 이후 서비스 사이클까지 같은 코드베이스에서 굴러가도록 만들기로 했다.

EVIDENCE · NOTES

- 단일 AMR 학습은 정착돼 있어도, 다대수 동시 학습에서는 정책 자체보다 자산 참조·미션 큐·환경 충돌 같은 비(非)알고리즘 문제에 더 많은 시간이 들어가는 패턴이 반복된다.
- 도킹까지는 학습이 잘 되더라도 도킹 이후의 retreat / returning 사이클이 정의돼 있지 않으면 다음 작업으로 연결되지 않아, 시뮬레이션이 "한 번 도킹하고 끝"으로 멈춘다.

02 SNAPSHOT

MERGED MR

50건

실험 환경 · 배포 인프라 · 런타임 반영

스케일링 폭

6 → 18대

동일 코드베이스 · config
변경만으로 분기

도킹 이후 서비스 사이클

3단계

retreat → returning →
done 큐 기반

03 ROLE

TEAM

5인 팀 (시뮬·학습·인프라
분담)

MY SCOPE

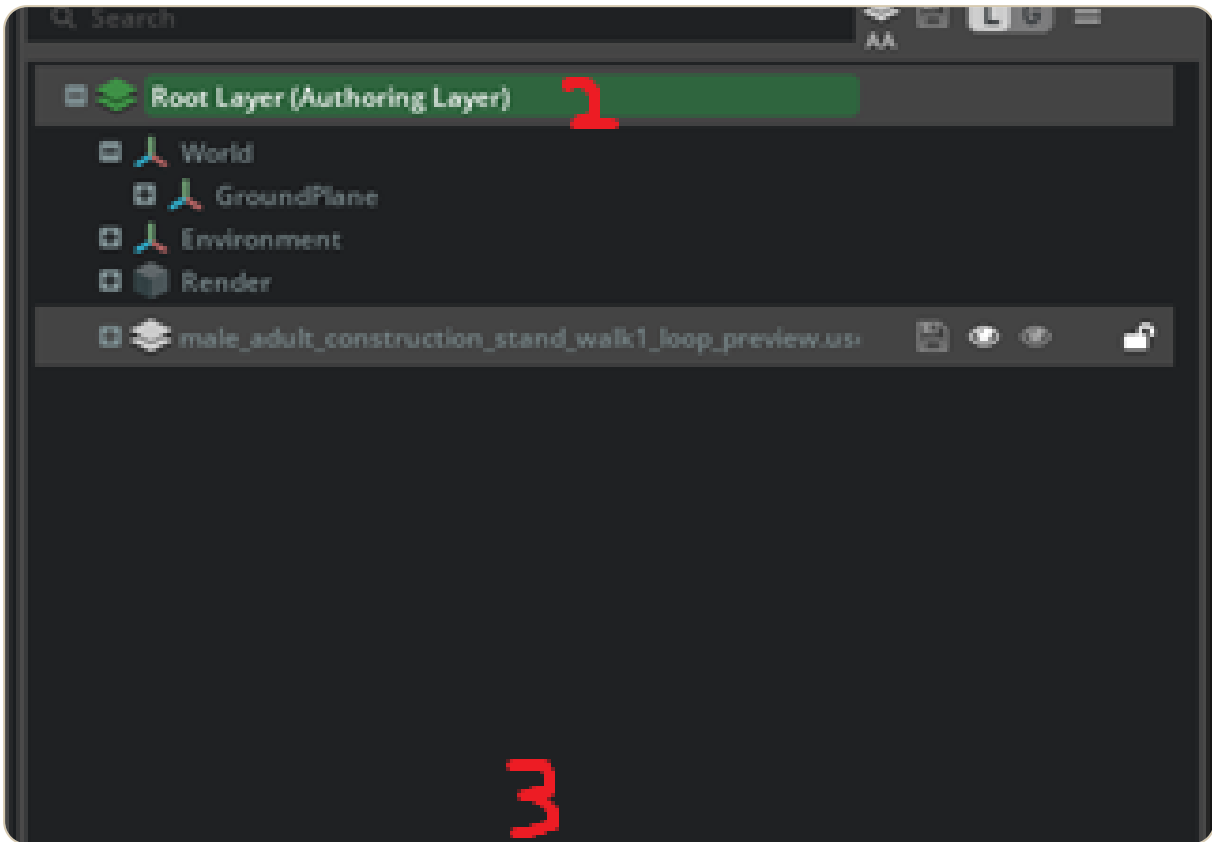
AMR navigation 학습 환경
(asset/config/scene/planner) 구성, 멀티 도킹 서비스 런타임, 스케일링 실험 환경, EC2/Caddy/Jenkins 배포 인프라

04 VISUALS

시뮬레이터 맵 캡처

SCREEN

실험 맵에 오버레이 자산을 엮어 시나리오를 구성할 때 사용한 실제 시뮬레이터 화면.



실험/배포 아키텍처

ARCH

학습 환경, 런타임, 텔레메트리, 배포 경로를 한 장으로 정리한 포트폴리오용 아키텍처.

AMR 시뮬레이션 플랫폼

Isaac Lab · multi-docking runtime · 6/12/18대 scaling · EC2/Caddy/Jenkins

Simulation Core

Isaac Lab

RL env · assets · scenes · planner

Runtime Loop

play_navigation_multi.p

dock → retreat → returning → done

Observability

Dashboard · Telemetry

status mapping · KPI snapshot

Training Variants

6 / 12 / 18

same codebase · config-only scale-up

Delivery Path

GitLab → Jenkins → Caddy → EC2

build, deploy, verify on the same GPU-friendly environment

05 PROBLEM SOLVING

문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

01

멀티 AMR 도킹 후 서비스 흐름이 끊겼다

! PROBLEM

도킹까지는 학습이 잘 되는데, 도킹 이후의 retreat / returning / done 사이클이 정의되지 않아 다음 작업으로 넘어가지 못했다.

→ APPROACH

play_navigation_multi.py를 추가하고 도킹 이후 retreat → returning → done 서비스 사이클을 큐 기반으로 정의했다.
service-v1 학습 환경을 함께 추가해 런타임과 학습 기준을 맞췄다.

✓ RESULT

도킹 이후에도 서비스 런타임이 끊기지 않는 3단계 후속 흐름을 만들었고, 멀티 도킹 시나리오를 끝까지 반복 실행할 기반을 확보했다.

↗ MR !110 · 멀티 도킹 서비스 런타임 및 service-v1 학습 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/110)

02

학습 환경을 늘릴 때마다 환경/경로 설정을 다시 뜯어야 했다

! PROBLEM

단일 AMR navigation 코드를 멀티대로 늘리면 asset/config 경로가 깨지고 scene이 충돌해, 실험 대수를 바꿀 때마다 코드를 복붙해야 했다. 다대수 학습 본 문제 이전에 환경 재구성이 발목을 잡는 패턴이 반복됐다.

→ APPROACH

isaacsim 경로를 rl_navigation 패키지로 재구성하고 실행 스크립트와 자산 참조를 새 레이아웃에 맞췄다. 패키지 단위로 분리해 N대 환경 추가가 config 변경만으로 가능하도록 만들었다. 단일·멀티 학습 환경을 같은 패키지에서 공유하도록 정리해 정책 코드와 실험 코드를 분리했다.

✓ RESULT

6/12/18대 스케일링 실험을 동일 코드베이스에서 분기 없이 실행할 수 있게 됐고, 환경 변경 비용이 사실상 0에 가깝게 줄어 학습 자체에 시간을 더 쓸 수 있게 됐다.

↗ MR !156 · AMR 최적대수 스케일링 실험 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/156)

! PROBLEM

Isaac Lab 학습 결과를 동료가 확인하려면 각자 로컬 환경이 맞춰야 했고, 학습 환경이 자주 변경되다 보니 "같은 코드인데 결과가 다르다"는 일이 반복됐다. GitHub Actions로는 GPU·Isaac Sim 의존을 다루기 어려워 CI에서 실험을 돌릴 수 없었다.

→ APPROACH

Jenkins를 EC2에 셀프호스팅으로 두고 build → deploy → verify를 같은 서버 맥락에서 돌리도록 묶었다. Caddy로 HTTPS 자동화를 붙여 실험 dashboard와 서비스 주소를 빠르게 정리했고, Docker 컨테이너 단위로 학습/런타임/대시보드를 격리했다.

✓ RESULT

학습 환경 변경이 코드 push로 곧장 배포·재현되는 흐름이 됐고, 팀원이 같은 셋업을 따로 빌드할 필요가 없어졌다. 환경 재현 비용이 "각자 로컬 셋업"에서 "머지 후 자동"으로 줄었다.

↗ MR !21 · Docker and EC2 deployment setup (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/21)

06 TECH DECISIONS

Isaac Lab

GPU 병렬 시뮬레이션과 RL 학습 툴체인이 한곳에 있어, 학습 환경과 플레이 테스트를 같은 문맥에서 돌릴 수 있었다. Gazebo/Webots 대비 다대수 환경에서 step throughput이 높고, 자산 참조·미션 큐를 패키지 단위로 정리하기 좋아 스케일링 실험에 유리했다.

rl_navigation 패키지화 + service-v1 학습 환경 분리

단일·멀티 학습 코드를 한 트리에 두면 변경 한 번에 모든 실험이 깨졌다. 정책 코드와 실험 환경 (scene/config/mission)을 패키지 경계로 분리하면 N대 추가가 config 변경만으로 끝나서, 학습 본 문제에 더 집중할 수 있었다.

Jenkins on EC2 (self-hosted)

Isaac Sim/학습 스택은 GPU·OS·드라이버 의존이 커서 GitHub Actions 같은 매니지드 러너로는 재현이 어려웠다. 셀프호스팅으로 build → deploy → verify를 같은 서버 맥락에서 돌려 "내 컴에 실행 됐는데" 문제를 구조적으로 줄였다.

Caddy reverse proxy

nginx 대비 HTTPS 자동화가 단순해서 실험 dashboard 주소를 빠르게 붙일 수 있었다. 운영 인프라 학습에 쓸 시간을 줄여, 인프라보다 실험 자체에 시간을 더 쓰는 트레이드오프를 의식적으로 택했다.

07 OUTCOMES

AMR 스케일링 실험

6 → 18대

같은 코드베이스로 대수만 바꿔 검증

도킹 이후 서비스 사이클

3단계

retreat → returning → done 큐 기반 자동 진행

학습 → 배포 파이프라인 통합

재현 자동화

Jenkins/Caddy/EC2로 push만으로 환경 재현

프로젝트 MERGED MR

50건

실험·인프라·연동 작업 누적 반영

08 SELECTED MRS

MR !110 · 멀티 도킹 서비스 런타임 및 service-v1 학습 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/110)

도킹 이후 서비스 사이클과 새 학습 환경을 함께 추가한 대표 MR.

MR !156 · 6/12/18대 스케일링 실험 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/156)

같은 코드베이스에서 다대수 실험을 돌릴 수 있게 한 확장 작업.

MR !21 · Docker and EC2 deployment setup (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/21)

로컬 검증과 EC2 배포 경로를 연결한 인프라 시작점.

09 RETROSPECTIVE

LESSON

다대수 RL은 정책 튜닝보다도 경로, 자산 참조, 미션 파일 구조가 먼저 흔들리기 쉬웠다. 학습 코드와 플레이 런타임의 경계를 초반에 잡는 일이 생각보다 더 중요했다.

NEXT TIME

다음에는 첫 주부터 env/config/log 포맷을 고정하고, 1대 → 6대 → 12대 smoke 시나리오를 자동화해 스케일 문제를 학습 전에 먼저 드러내겠다.

← Home

2026.01 - 2026.02 · 6주 · 공통PJT

itda

WebRTC로 다인 협업을 묶고 통합 대시보드 UI에서 룸·타임라인·세션을 한 번에 관리하는 AI 영상 제작 협업 플랫폼.

Vue.js

WebRTC

Spring Boot

Node.js

Docker

S3

01 BACKGROUND

PROBLEM

AI 영상 제작 협업은 시나리오 작성, 씬 편집, 타임라인 정리, 실시간 회의가 보통 서로 다른 도구에 흩어져 있다. 그러다 보니 협업 중간에 도구를 옮겨 다니면서 맥락이 끊기고, 누가 무엇을 보고 있는지가 화면 사이에서 사라진다. 우리 팀은 시나리오·씬·타임라인·화상 통화를 한 화면 안에서 굴리는 협업 공간이 필요하다고 봤다.

OUR APPROACH

Vue 기반 통합 워크스페이스에 chat·presence·rtc 3개 협업 채널을 projectId 기준으로 묶고, MiniTimeline과 사이드바를 공통 셀로 정리해 시나리오 → 씬 편집 → 회의가 같은 화면 계층에서 연결되도록 만들기로 했다.

EVIDENCE · NOTES

- 기존 영상 제작 협업 흐름은 시나리오 도구(문서) → 편집 도구(타임라인) → 회의 도구(화상)가 분리돼 있어, 같은 프로젝트인데도 사용자가 여러 탭을 동시에 띄우고 맥락을 머릿속에서 합쳐야 했다.
- WebRTC 기반 소규모 협업은 SFU 운영 비용 없이 P2P로도 6명 수준까지 잘 굴러간다는 일반 관찰이 있어, 통합 협업 공간의 MVP를 P2P로 시작하는 게 현실적이었다.

02 SNAPSHOT

실시간 협업 구조

3채널

chat · presence · rtc를
projectId 기준 통합

WEBRTC MESH 통화 범위

6명

P2P 전제로 문서/UI/코드
스코프 통일

사이드바 +
MINITIMELINE 통합

공통 셀

대시보드·상세·에디터가 같
은 협업 패널 슬롯 공유

03 ROLE

TEAM

6인 팀

MY SCOPE

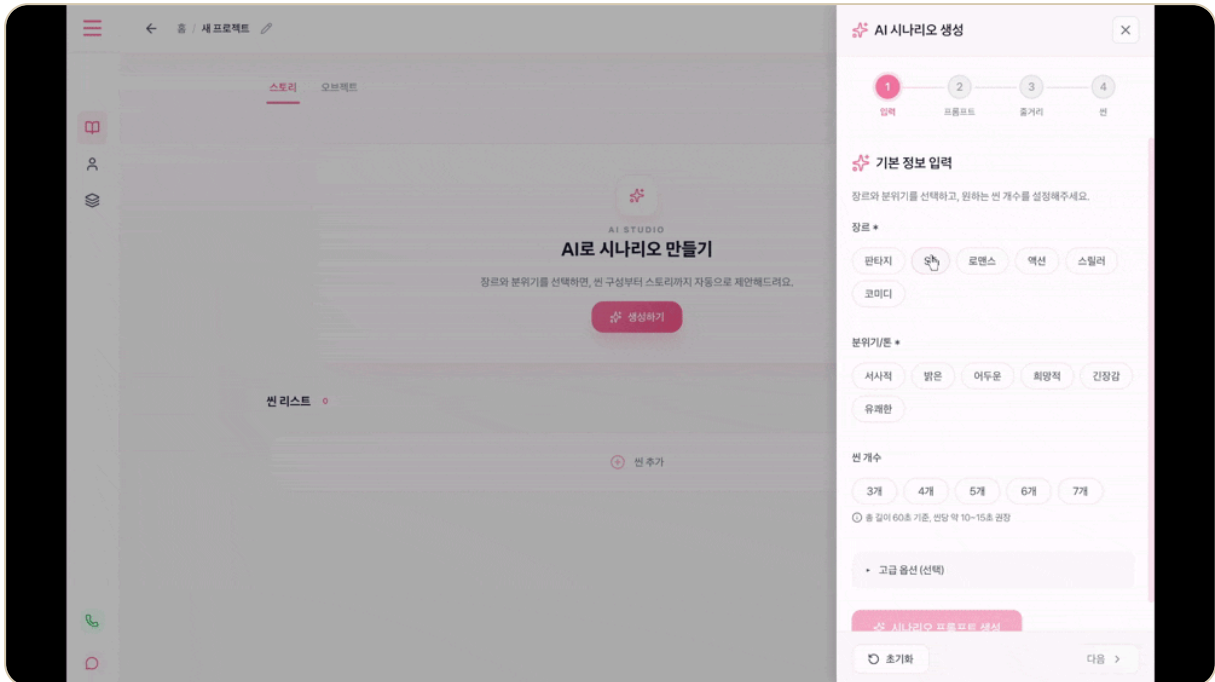
WebRTC 시그널링 처리, 사이드바·MiniTimeline UI 통합,
라우팅 정리

04 VISUALS

AI 시나리오 생성

장르, 분위기, 씬 개수를 입력해 시나리오를 생성하는 실제 동작 GIF.

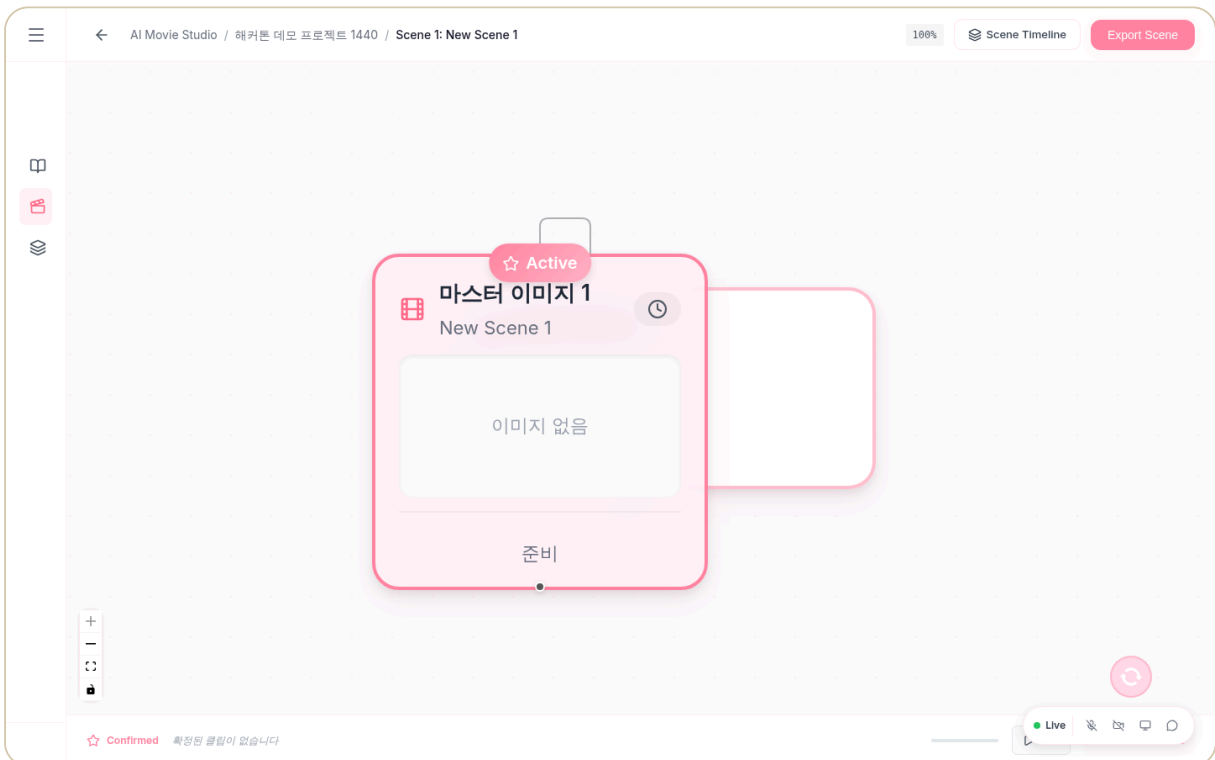
GIF



씬 에디터

MiniTimeline과 협업 UI가 들어가는 핵심 작업 공간. 노드 기반 편집의 중심 화면이다.

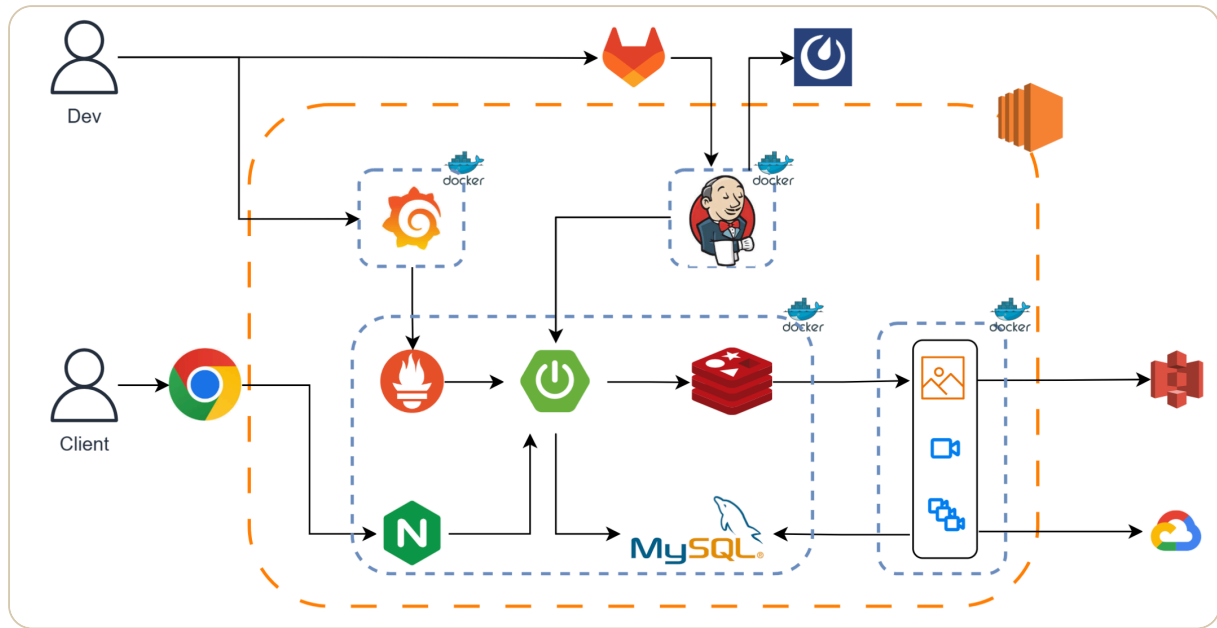
SCREEN



시스템 아키텍처

프로젝트 README에 포함돼 있던 실제 아키텍처 이미지. 협업, AI 작업, 배포 흐름이 한 장에 담겨 있다.

ARCH



05 PROBLEM SOLVING

문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

! PROBLEM

프로젝트 상세, 씬 편집, 타임라인 레이아웃이 각각 소켓·프레즌스·RTC 연결을 따로 품으면 페이지 이동 때 구독 해제와 재연결 순서가 어긋나 협업 바가 쉽게 불안정해졌다. WebRTC peer 연결이 라우트 전환마다 재생성돼 미디어 스트림이 끊기는 회귀도 잦았다.

→ APPROACH

collabStore를 중심으로 chat / presence / rtc 3개 채널을 projectId 기준으로 묶고, socketManager와 peerConnectionService의 join → subscribe → media 초기화 순서를 명시적으로 고정했다. 레이아웃은 공통 사이드바와 협업 채널 슬롯을 공유해, 페이지가 바뀌어도 채널 구독이 끊기지 않도록 정리했다.

✓ RESULT

대시보드·상세·에디터에서 같은 협업 상태를 재사용할 수 있는 구조를 만들었고, 최대 6명 Mesh 전제를 코드와 문서 양쪽에 맞춰 정렬했다. peer 재연결로 인한 미디어 끊김 회귀가 정리됐다.

↗ 협업 채널 3종(chat/presence/rtc) + 6명 Mesh 설계

! PROBLEM

MiniTimeline, 사이드바, 프로젝트/씬 편집 레이아웃을 여러 브랜치가 동시에 건드리면서 클릭 흐름과 URL 파라미터가 엇갈릴 때 화면 전환이 쉽게 깨졌다. 데모 직전 회귀가 이 영역에서 반복돼 안정화 시간이 길어졌다.

→ APPROACH

공통 레이아웃과 라우터 경계를 다시 나누고, projectId/scenelId 파라미터를 기준으로 페이지 책임을 정리했다. 충돌이 잦았던 공통 셀 영역은 merge 단위를 잘게 쪼개 반영해 회귀 범위를 좁혔고, 데모 안정화 MR(!69)로 흐름을 한 번 되돌린 뒤 다시 통합했다.

✓ RESULT

프로젝트 상세와 씬 편집 진입 흐름을 다시 안정화했고, 협업 UI와 MiniTimeline을 같은 화면 계층에 올릴 수 있게 됐다. 데모 직전 회귀의 원인 범위를 좁혀 마무리 안정화 비용을 줄였다.

↗ MR !70 · 라우팅 수정 (https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205/-/merge_requests/70)

06 TECH DECISIONS

WebRTC P2P (Mesh)

최대 6명 통화를 MVP 범위로 잡았기 때문에 SFU 서버를 따로 운영할 만한 트래픽이 아니었다. P2P Mesh는 시그널링만 우리 서버를 거치고 미디어는 클라이언트끼리 직접 주고받아, 운영 비용·인프라 복잡도를 낮추는 대신 인원 확장은 포기하는 트레이드오프였다.

Vue.js + Pinia (collabStore)

편집기 상태와 협업 상태(채널 구독·peer 연결·미디어 트랙)를 분리해서 다뤄야 했다. Composition API + Pinia 조합은 store 단위로 책임을 자르기 쉬워, 페이지가 바뀌어도 협업 store 하나가 채널 라이프사이클을 들고 있는 구조를 만들 수 있었다.

STOMP presence + 채널 라우팅 분리

chat은 브로드캐스트, presence는 1:1 큐, rtc는 시그널링 메시지처럼 채널마다 라우팅 패턴이 달랐다. 같은 소켓 위에서 STOMP destination으로 나눠두면 레이아웃이 달라도 협업 UI를 같은 규칙으로 재사용할 수 있었다.

07 OUTCOMES

CHAT · PRESENCE · RTC 라
이프사이클

3채널 통 합

projectId 기준으로 묶고
join → subscribe →
media 순서 고정

사이드바 +
MINITIMELINE 슬롯

공통 셀 1 개

대시보드·상세·에디터가 같
은 협업 패널을 공유

MESH 통화 설계 범위

6명

문서/코드/UI 스코프를 동일
기준으로 통일

라우팅 회귀 정리

데모 안정 화

MR !70!/69로 화면 전환 흐
름을 다시 잡음

08 SELECTED MRS

MR !70 · 라우팅 수정 (https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205/-/merge_requests/70)

프로젝트/씬 편집 전환이 흔들리던 경로를 다시 맞춘 정리 MR.

MR !69 · 원상복구 (https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205/-/merge_requests/69)

데모 직전 흐름 안정화와 UI 회귀를 되돌린 보수 MR.

09 RETROSPECTIVE

LESSON

실시간 협업은 미디어 처리보다도 이벤트 라우팅과 재접속 순서를 먼저 고정해야 UI가 안정해진다. 화면은 그 위에 올라오는 결과물에 가깝다.

NEXT TIME

다음에는 collab contract(chat/presence/rtc)와 reconnect 시나리오를 첫 주부터 fixture로 만들고, 레이아웃 공통 셀 변경은 별도 브랜치로 격리하겠다.

[다른 프로젝트 보기](#)

GitLab repo ↗ (<https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205>)

← Home

2025.12 · 1주 · 관통PJT

MovieApp

영화 추천·한줄평·마이페이지를 갖춘 영화 커뮤니티. 첫 풀스택 협업 프로젝트로, 사용자 콘텐츠 흐름(좋아요·한줄평·나중에 볼)을 끝에서 끝까지 묶는 게 과제였다.

Vue.js

Django REST Framework

TMDB API

01 BACKGROUND

PROBLEM

1주짜리 첫 풀스택 협업 프로젝트로, 영화 데이터를 활용해 사용자가 자기 취향을 정리·공유하는 공간을 만드는 게 과제였다. 단순 카탈로그가 아니라 "내 활동이 모이는 곳"이 되려면, 좋아요·한줄평·나중에 볼 같은 사용자 행동이 한 화면에서 연결돼야 했다.

OUR APPROACH

TMDB로 영화 메타데이터 부담을 외부 API에 위임하고, 사용자 활동(좋아요·한줄평·나중에 볼)은 watchlist 관계 모델로 묶어 마이페이지 한 화면에 모은다. 추천 영역은 사용자 반응 신호를 가볍게 활용해 단순 인기 목록과 차별화한다.

EVIDENCE · NOTES

- 영화 추천/메타데이터 서비스는 많지만, 한줄평·좋아요·나중에 볼 같은 사용자 활동이 한 마이페이지에 통합돼 있지 않으면 "내 취향 기록"이 화면 사이에 흩어진다는 일반적 관찰이 있었다.
- 1주 단위 짧은 프로젝트라서, 메타데이터 수집·관리에 시간을 쓰는 대신 사용자 행동 흐름과 협업 워크플로우(브랜치/MR) 자체를 배우는 데 시간을 더 쓰는 편이 맞았다.

02 SNAPSHOT

직접 맡은 사용자 기능

4흐름

마이페이지 · 한줄평 · 나중
에 볼 · 추천

MERGED MR (1주)

5건

첫 풀스택 협업의 기능 단위
사이클

관통PJT 기간

1주

첫 풀스택 협업 + Vue/DRF
학습 동시 진행

03 ROLE

TEAM

3인 팀

MY SCOPE

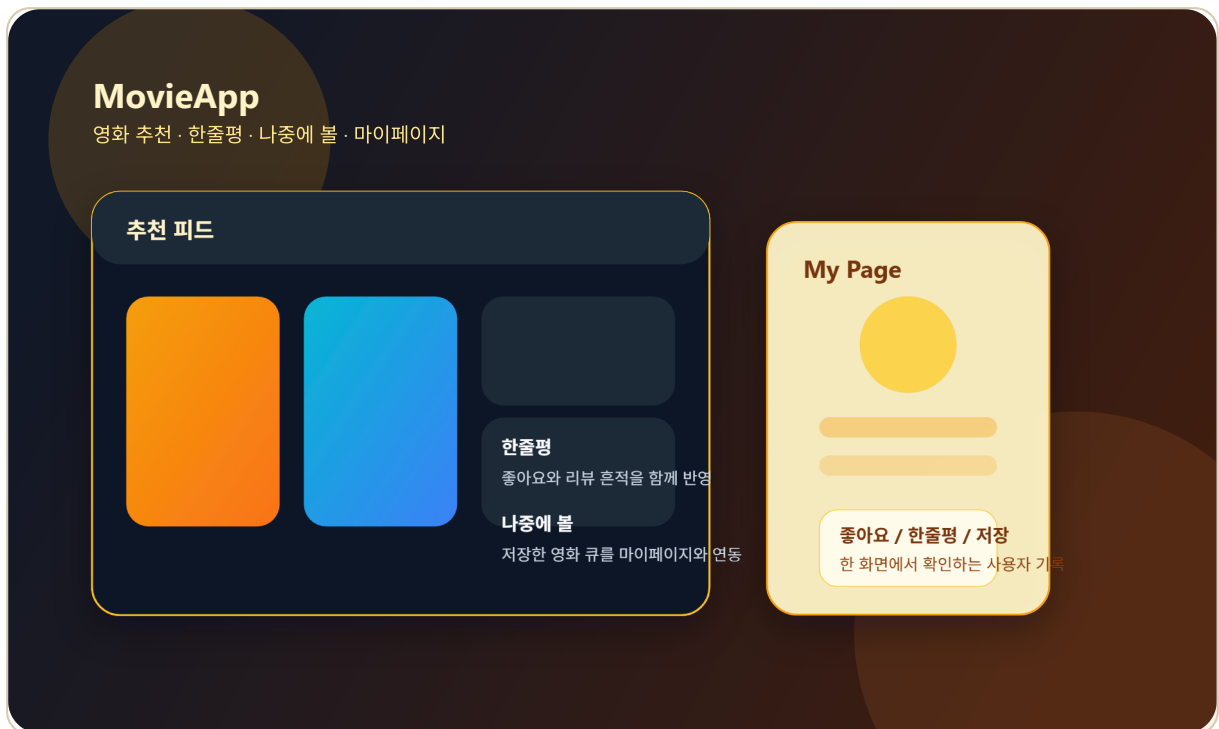
마이페이지(프로필/좋아요/한줄평), 한줄평 작성·조회, 추천
콘텐츠 큐

04 VISUALS

대표 화면 보드

실제 화면 캡처가 남아 있지 않아, 추천 피드와 마이페이지 중심으로 기능 구성을 다시 정리한 포트폴리오용 보드.

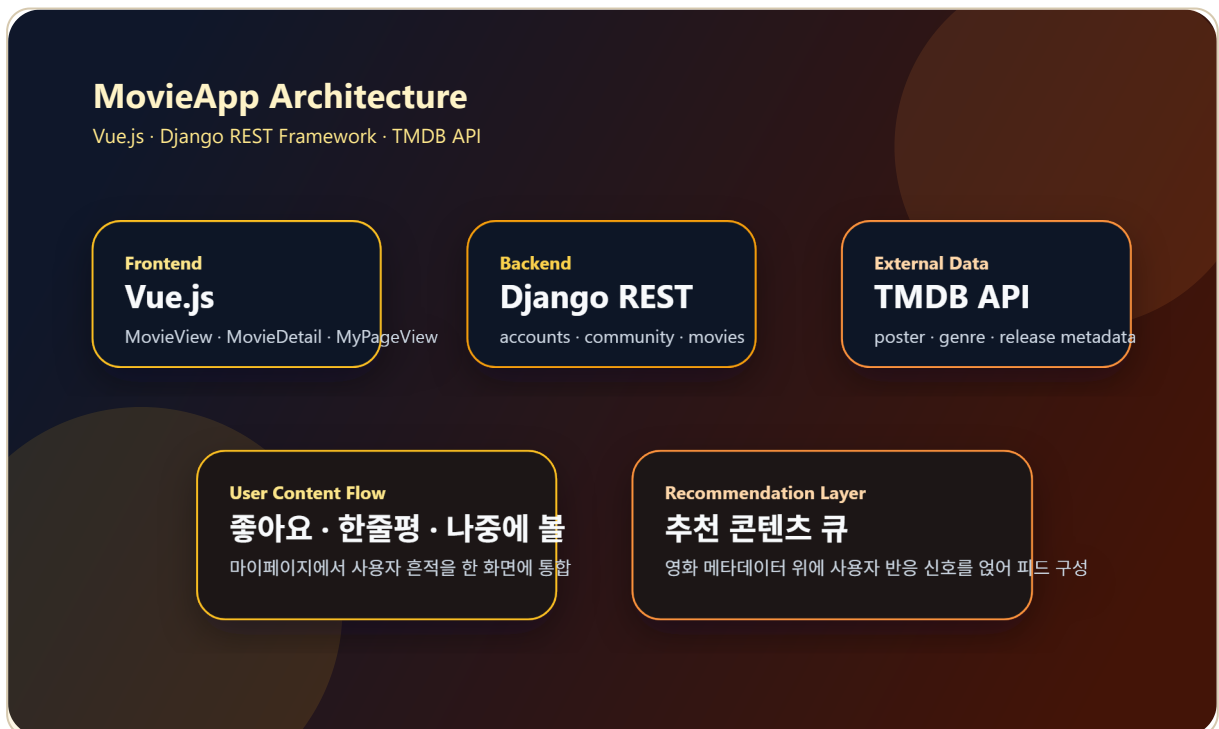
PREVIEW



서비스 구성

Vue 프론트, DRF 백엔드, TMDB 메타데이터 연동, 사용자 콘텐츠 흐름을 설명한 아키텍처.

ARCH



문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

01

사용자 활동이 화면 사이에 흩어져 마이페이지가 비어 보였다



PROBLEM

좋아요·한줄평·나중에 볼 기능이 영화·커뮤니티·계정 화면에 따로 살아 있어, 사용자가 자기 활동을 한눈에 모아 보지 못했다. 마이페이지가 프로필 정보만 보여주고 정작 "내 흔적"이 비어 있는 구조였다.



APPROACH

watchlist 관계를 모델·migration에 추가하고, movies/community/account 뷰를 묶어 MyPageView에서 프로필·저장 목록·한줄평·흐름을 함께 노출하도록 정리했다. 1주 일정이라 데이터 모델을 먼저 못 박고 화면을 그 위에 얹는 순서로 진행했다.



RESULT

마이페이지가 사용자 활동 허브가 됐고, 좋아요/한줄평/나중에 볼 흐름이 같은 화면에서 연결되며 "내 활동이 모이는 곳"이라는 기획 의도가 그대로 드러나는 구조가 됐다.

➤ MR !7 · 마이페이지 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/7)

추천 영역이 단순 인기 목록이라 커뮤니티 서비스다운 차별점이 약했다

! PROBLEM

메인 피드가 TMDB 기준 최신·인기 목록만 노출하면 어느 영화 사이트와도 다르지 않았다. 커뮤니티 서비스라면 "이 사용자에게 어울리는 것"이 보여야 했지만, 그 자리가 비어 있었다.

→ APPROACH

MovieView 메인과 백엔드 movies/views.py에서 추천 영역을 분리하고, 좋아요·한줄평·나중에 볼 같은 사용자 반응 신호를 추천 큐에 반영하도록 구성했다. 추천 알고리즘 자체는 가벼운 규칙 기반으로 두고, "신호 → 추천 큐" 흐름을 먼저 만드는 데 집중했다.

✓ RESULT

메인 피드에 개인화된 추천 블록이 추가되며 단순 카탈로그와 구분되는 탐색 흐름이 만들어졌다.

↗ MR !13 · 좋아할 만한 콘텐츠 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/13)

06 TECH DECISIONS

Vue.js + Django REST Framework

1주 안에 화면과 API를 동시에 붙여야 했다. Vue는 컴포넌트 단위로 화면을 빠르게 쪼개기 좋았고, DRF는 ORM + serializer로 영화·커뮤니티·계정 도메인의 CRUD를 빠르게 묶어 낼 수 있었다. "화면-라우터-시리얼라이저"가 거의 일대일로 붙는 구조라 첫 협업의 학습 곡선이 가팔라지지 않았다.

TMDB API (외부 메타데이터 위임)

포스터·장르·개봉일 같은 영화 메타데이터를 직접 적재·관리하면 1주 일정이 전부 거기로 빨려 들어간다. 외부 API에 위임하면 그 시간을 사용자 행동 기능(좋아요·한줄평·나중에 볼)과 협업 워크플로우 학습에 돌릴 수 있다는 트레이드오프가 분명했다.

watchlist 관계 모델 ("내 활동" 통합 단위)

마이페이지를 "내 활동이 모이는 곳"으로 만들려면 좋아요·한줄평·나중에 볼이 같은 단위에서 조회돼야 했다. watchlist 관계를 명시적인 모델로 두면 화면이 늘어도 같은 쿼리 패턴으로 활동을 모을 수 있어, 1주짜리 프로젝트에서도 확장의 여지를 남길 수 있었다.

07 OUTCOMES

사용자 활동 허브

4흐름 통 합

마이페이지 · 한줄평 · 나중
에 볼 · 추천이 한 화면 계층
에 연결

활동 통합 모델 1개

watchlist

좋아요·저장·한줄평이 같은
관계 단위로 조회

MERGED MR (1주)

5건

기능 단위로 잘게 쪼개 반영
한 첫 풀스택 협업 사이클

08 SELECTED MRS

MR !7 · 마이페이지 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/7)

watchlist 관계와 MyPageView를 포함해 사용자 활동 허브를 만든 대표 MR.

MR !12 · 한줄평 등등 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/12)

영화 상세와 라우팅을 건드리며 리뷰 작성/조회 흐름을 붙인 작업.

MR !13 · 좋아할 만한 콘텐츠 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/13)

메인 영화 피드에 추천 블록을 추가한 마무리 기능.

09 RETROSPECTIVE

LESSON

첫 협업에서는 화면보다 데이터 모델을 먼저
맞추는 편이 전체 속도를 더 높인다. 사용자

NEXT TIME

다음에는 관계형 필드와 응답 shape를 먼저
문서화하고, 라우터와 뷰 이름까지 초반에 맞

활동이 여러 화면에 걸치면 관계 설계가 늦을
수록 프론트와 백이 함께 흔들린다.

춘 뒤 구현하겠다.

[다른 프로젝트 보기](#)

[GitLab repo ↗](#) (<https://lab.ssafy.com/ljin2091/final-pjt>)